Implementing Data Privacy

70502, 72298

November 2, 2025

Introduction

No âmbito da cadeira de Privacidade de Sistemas de 2025/2026, foi-nos pedido pelo professor Kevin Gallagher, como primeiro projeto do semestre, que fossem implementadas as técnicas de privatização de dados de utilizadores estudadas em aula - **Data De-Identification, *k*-Anonymity e Differential Privacy** - e que fosse realizado um **Linkage Attack**, de forma a averiguar o impacto destas técnicas na privacidade dos utilizadores. Para resultados realísticos, foi usada uma base de dados em *Docker* com dados simulados, um cliente e um servidor.

Data Privacy Techniques Implemented

Data De-Identification

Com esta técnica, surge a criação de uma tabela nova na base de dados a que chamámos *med_data_de_id*, que consiste numa cópia da tabela de *med_data* mas com alguns atributos removidos: o *name*, a *address* e o *email*. A esta técnica dá-se o nome de *Suppression*, que consiste em remover a informação da base de dados que permita a um atacante identificar um membro da mesma 1.

Linkage Attack

De forma a comparar como é que estas técnicas de privatização impactam a anonimização da base de dados, foi-nos pedido que implementássemos um mecanismo de *Linkage Attack*, que consiste em correlacionar dados entre diferentes bases de dados para identificação de entradas. Para isto, criámos uma nova classe Java com o método 2 que itera sobre as entradas da tabela *work_data* e para cada uma, compara-a com as entradas na tabela *med_data_de_id*, de forma a obter possíveis *matches*, devolvendo-os no final da execução (removendo os repetidos). Para comparar se são realmente possíveis *matches*, usámos apenas uma comparação simples sobre os atributos que mais facilmente identificavam uma entrada quando juntos, o *gender* e o *postalCode* 3.

k-Anonymity

Como dado nas aulas, uma forma de anonimização dos dados é por *k*-Anonymity, que utiliza técnicas - *Suppression*, *Generalization*, e *Top and Bottom Coding* - de forma a que haja pelo menos *k* entradas indistinguíveis pelos seus atributos. Este valor é automaticamente atribuído a partir do documento *application.properties*.
Para implementar esta técnica, criámos uma nova tabela *med_data_k* a partir da *med_data_de_id*

4 e programámos o *Mondrain Algorithm* 5, que é responsável pela divisão das entradas em subconjuntos de forma a que cada um destes tenha entradas indistinguíveis. Como é possível analisar no código, fazemos *Generalization* dos valores do *postalCode* para cada subconjunto da tabela, colocando todas as entradas do subconjunto entre o menor e o maior desse mesmo conjunto. Para o *gender*, aplicámos a técnica de *Suppression* e transformámos o *gender* da tabela inteira em *. Após a *k*-Anonymization da tabela, criámos um método novo de *Linkage Attack* que tenta correlacionar os dados entre as tabelas *work_data* e *med_data_k* 6, usando um método de comparação que analisa todos os casos possíveis de códigos postais da tabela 7.

Differential Privacy

Para implementar esta técnica criámos a classe Java *MedDifferentialResponse*, que é responsável pela simulação das *Queries* feitas pelo *Client* à base de dados, sejam elas *Queries* para contar os membros com um certo atributo, somas, médias ou *grouped counts*, que são feitas a partir da classe Java *MedDataRepository*, com algumas *Queries* implementadas de exemplo 8. Na classe que implementa a *Differential Privacy*, usamos a distribuição de Laplace para adicionar *noise* às *Queries* feitas 9, que atualiza automaticamente o *security parameter* de acordo com o número de *Queries* feitas à base de dados. O *treshold* é definido a partir do documento *application.properties*.

Conclusion

Com este projeto, pudemos observar o quão úteis são as técnicas de privatização de dados - e quão difíceis são de implementar - a partir dos resultados obtidos da função de *Linkage Attack* após cada técnica implementada que, como esperado, eram cada vez menos úteis para o atacante. Tal como Daniel Keys Moran afirmou,

> ”You can have data without information, but you cannot have information without data.”
> *- Daniel Keys Moran*

```python
with open("./Data/med.csv", 'r') as med_file:
    med_csv = csv.reader(med_file)

    for line in med_csv:
        row = MED_TABLE.insert().values(id=line[0], name=line[1], age=line[2],
            address=line[3], email=line[4], gender=line[5], postal_code=line[6],
            diagnosis=line[7])
        CONN.execute(row)
    CONN.commit()

with open("./Data/med.csv", 'r') as med_file:
    med_csv = csv.reader(med_file)

    for line in med_csv:
        row = MED_TABLE_DEID.insert().values(id=line[0], age=line[2],
            gender=line[5], postal_code=line[6], diagnosis=line[7])
        CONN.execute(row)
    CONN.commit()
```

Figure 1: Diferenças no *script* que provocam a criação da tabela *De-Identified*.

```java
public int linkageAttackDeId() {
    List<WorkData> workData = workDataRepository.findAll();
    List<MedDataDeId> medData = medDeIdDataRepository.findAll();

    List<String> matches = new ArrayList<>();

    for (WorkData work : workData) {
        for (MedDataDeId med : medData) {
            if (work.isLinkedDeId(med)) {
                String nameFull = work.getFName().concat(" ").concat(work.getLName());
                if (!matches.contains(nameFull))
                    matches.add(nameFull);
            }
        }
    }
    return matches.size();
}
```

Figure 2: Método *linkageAttackDeId*, na classe Java *LinkageAttack*.

```java
public boolean isLinkedDeId(MedDataDeId other) {
    return this.gender.equals(other.getGender()) && this.postalCode.equals(other.getPostalCode());
}
```

Figure 3: Método *isLinkedDeId*, na classe *WorkData*.

```python
with open("./Data/med.csv", 'r') as med_file:
    med_csv = csv.reader(med_file)

    for line in med_csv:
        row = MED_TABLE_DEID.insert().values(id=line[0], age=line[2],
            gender=line[5], postal_code=line[6], diagnosis=line[7])
        CONN.execute(row)
    CONN.commit()

with open("./Data/med.csv", 'r') as med_file:
    med_csv = csv.reader(med_file)

    for line in med_csv:
        row = MED_TABLE_K.insert().values(id=line[0], age=line[2],
            gender=line[5], postal_code=line[6], diagnosis=line[7])
        CONN.execute(row)
    CONN.commit()
```

Figure 4: Criação da tabela *med_data_k* a partir do *script* em *Python*.

```java
public List<MedDataK> mondrainAlgorithm(List<MedDataK> medDataKRecursive){
    int listSize = medDataKRecursive.size();

    //Base Case
    if (listSize <= getBaseEntrySize()) {
        int maxPostalCode   = Integer.MIN_VALUE;
        int minPostalCode   = Integer.MAX_VALUE;

        for (MedDataK dataEntry : medDataKRecursive){
            dataEntry.setGender(gender: "*");

            int postalCodeInt = Integer.parseInt(dataEntry.getPostalCode());

            maxPostalCode = (postalCodeInt > maxPostalCode) ? postalCodeInt : maxPostalCode;
            minPostalCode = (postalCodeInt < minPostalCode) ? postalCodeInt : minPostalCode;
        }
        for (MedDataK dataEntry : medDataKRecursive){
            dataEntry.setPostalCode(Integer.toString(minPostalCode) + "-" + Integer.toString(maxPostalCode));
        }

        return medDataKRecursive;
    }
    else {
        //Break the partition in half
        List<MedDataK> leftSideMedDataK     = medDataKRecursive.subList(0, medDataKRecursive.size() / 2);
        List<MedDataK> rightSideMedDataK    = medDataKRecursive.subList(medDataKRecursive.size() / 2, medDataKRecursive.size());

        //call the Mondrain algorithm on the left and right half of the partition
        List<MedDataK> leftSideMedDataKResult   = mondrainAlgorithm(leftSideMedDataK);
        List<MedDataK> rightSideMedDataKResult  = mondrainAlgorithm(rightSideMedDataK);

        //Return the Union of the result
        List<MedDataK> unionOfResults = new ArrayList<>(leftSideMedDataKResult);
        unionOfResults.addAll(rightSideMedDataKResult);

        return unionOfResults;
    }
}
```

Figure 5: Método *mondrainAlgorithm* na classe Java *MondrainAlgorithm*.

```java
public int linkageAttackK() {
    List<WorkData> workData = workDataRepository.findAll();
    List<MedDataK> medData = medDataKRepository.findAll();

    List<String> matches = new ArrayList<>();

    for (WorkData work : workData) {
        for (MedDataK med : medData) {
            if (work.isLinkedK(med)) {
                String nameFull = work.getFName().concat(" ").concat(work.getLName());
                if (!matches.contains(nameFull))
                    matches.add(nameFull);
            }
        }
    }
    return matches.size();
}
```

Figure 6: Método de *Linkage Attack* para a tabela *k*-anonimizada, na classe Java *LinkageAttack*.

```java
public boolean isLinkedK(MedDataK other) {
    // has letters
    String[] otherPostalCodeInterval = other.getPostalCode().split("-");

    int otherPostalCodeMin = Integer.valueOf(otherPostalCodeInterval[0]);
    int otherPostalCodeMax = Integer.valueOf(otherPostalCodeInterval[1]);

    String[] hifenPostalCode = this.postalCode.split("-");
    String[] spacePostalCode = this.postalCode.split(" ");

    if (hifenPostalCode.length == 2) { // has -, como 11900-000
        int thisPostalCodeFirst = Integer.parseInt(hifenPostalCode[0]);
        int thisPostalCodeSecond = Integer.parseInt(hifenPostalCode[1]);

        if (thisPostalCodeFirst >= otherPostalCodeMin && thisPostalCodeFirst >= otherPostalCodeMin)
            return thisPostalCodeSecond >= otherPostalCodeMin && thisPostalCodeSecond <= otherPostalCodeMax;
        else
            return false;
    } else if (spacePostalCode.length == 2) { // has space, como 92980 CEDEX ou 755 01
        int thisPostalCodeNumber = Integer.parseInt(spacePostalCode[0]);

        return thisPostalCodeNumber >= otherPostalCodeMin && thisPostalCodeNumber >= otherPostalCodeMin;
    } else if (spacePostalCode.length == 3) { // has two spaces, como 75712 CEDEX 15
        int thisPostalCodeNumber = Integer.parseInt(spacePostalCode[0]);

        return thisPostalCodeNumber >= otherPostalCodeMin && thisPostalCodeNumber >= otherPostalCodeMin;
    } else if (this.postalCode.isBlank()) { // is blank
        return true;
    } else if (Character.isLetter(this.postalCode.charAt(0))) { //has letters, como L9T
        return false;
    }

    int thisPostalCode = Integer.parseInt(this.postalCode);

    if (thisPostalCode >= otherPostalCodeMin && thisPostalCode <= otherPostalCodeMax)
        return true;
    else
        return false;
}
```

Figure 7: Método *isLinkedK*, na classe *WorkData*.

```java
@Repository
public interface MedDataRepository extends JpaRepository<MedData, Long> {
    // Custom query methods can be defined here
    long countByName(String name);
    long countByAge(int age);
    long countByAddress(String address);
    long countByEmail(String email);
    long countByGender(String gender);
    long countByPostalCode(String postalCode);
    long countByDiagnosis(String diagnosis);

    @Query("SELECT COUNT(md.age) FROM MedData md WHERE md.age IS NOT NULL")
    int getTotalAgeDataPoints();

    @Query("SELECT SUM(md.age) FROM MedData md")
    Long getTotalAge();

    @Query("SELECT COUNT(md.age) FROM MedData md WHERE md.diagnosis = :diagnosis")
    int getTotalAgeDataPointsByDiagnosis(@Param("diagnosis") String diagnosis);

    @Query("SELECT SUM(md.age) FROM MedData md WHERE md.diagnosis = :diagnosis")
    int getSumAgeByDiagnosis(@Param("diagnosis") String diagnosis);

}
```

Figure 8: Classe Java *MedDataRepository*, com as *Queries* de exemplo mencionadas.

```java
private double laplace(int sens) {
    double x = sens / laplaceConfig.getLaplace();
    LaplaceDistribution laplace = new LaplaceDistribution(0, x);
    laplaceConfig.setLaplace(laplaceConfig.getLaplace() + CONFIDENTIALITY_LOSS);
    setBudget(laplaceConfig.getLaplace());
    return laplace.sample();
}
```

Figure 9: Método da Distribuição de Laplace.